EXPERIMENT 7
EMAIL NOTIFICATION

Spring Boot + PostgreSQL + Gmail SMTP + JavaMailSender

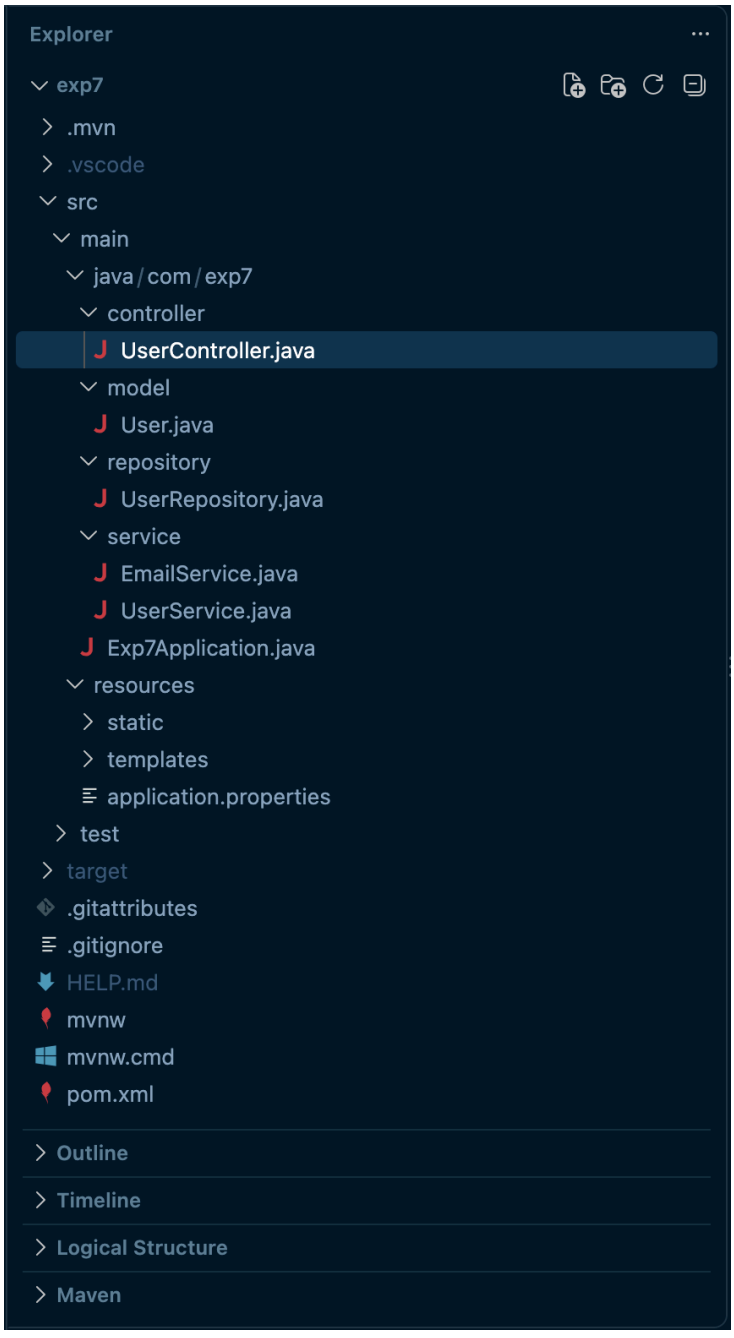
1. Aim

To implement an email notification system in Spring Boot that sends a confirmation email to a registered user after successful signup using JavaMailSender and Gmail SMTP.

2. Requirements

- Spring Boot project
- Java 21
- Spring Data JPA
- Spring Mail
- Spring Web MVC
- PostgreSQL
- Gmail account with 2-Step Verification and App Password
- Postman

3.Project Structure



4. Step-by-Step Implementation

Step 1: Create the Spring Boot Project in VS Code

1. Open **Visual Studio Code**.
2. Press: Ctrl + Shift + P

3. In the Command Palette, search for: Spring Initializr: Create a Maven Project
4. Select: Spring Initializr: Create a Maven Project
5. Select the project language: **Java**
6. Select the build tool: **Maven**
7. Select the Spring Boot version available in the Spring Initializr.
   For this experiment, the project was created using: **Spring Boot 4.1.1**
8. Enter the project details: **Group Id: com.exp7** , **Artifact Id: exp7**
9. Select: Packaging: **Jar**
10. Select the Java version: **Java 21**
11. Add the following dependencies: **Spring Data JPA, Spring Mail, Spring Web MVC, PostgreSQL Driver.**
12. Select the location where the project should be created.
13. Enter/select the project name: **exp7**
14. VS Code generates the Spring Boot project with the required Maven structure.
15. Open the generated `exp7` project in VS Code.

The basic project structure is:

```
exp7
├── src
│   ├── main
│   │   ├── java
│   │   └── resources
│   └── test
├── pom.xml
└── mvnw
```

16. Open the `pom.xml` file and verify that the selected dependencies have been added successfully.

Replace Entire **pom.xml** with this following code:

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-parent</artifactId>

        <version>4.1.1</version>

        <relativePath/>

    </parent>

    <groupId>com.exp7</groupId>

    <artifactId>exp7</artifactId>

    <version>0.0.1-SNAPSHOT</version>

    <name>exp7</name>

    <description>Experiment 7 - Email Notification</description>

    <properties>

        <java.version>21</java.version>

    </properties>

    <dependencies>

        <!-- Spring Data JPA -->

        <dependency>

            <groupId>org.springframework.boot</groupId>

            <artifactId>spring-boot-starter-data-jpa</artifactId>

        </dependency>
```

```
<!-- Email Support -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>

<!-- Spring Web -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>

<!-- PostgreSQL Database -->
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- Test Dependencies -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa-test</artifactId>
    <scope>test</scope>
</dependency>
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-mail-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>

    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Step 2: Create the PostgreSQL Database

A PostgreSQL database named exp7_db was created using pgAdmin.

2.1 Open pgAdmin

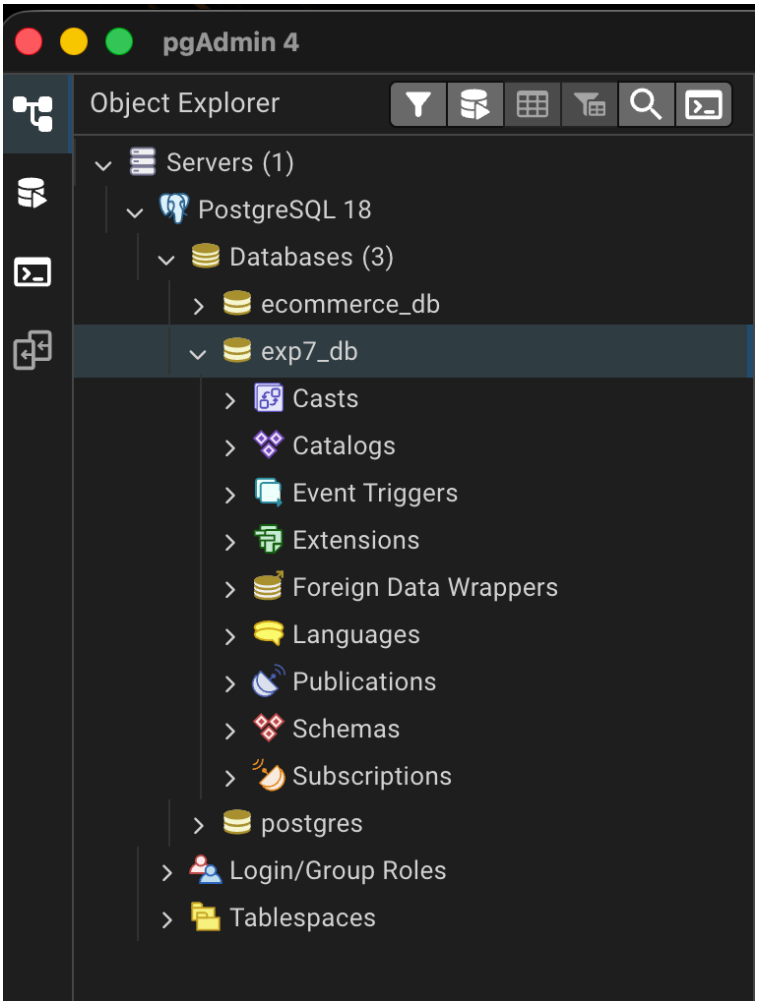
1. Open **pgAdmin** on the system.
2. Connect to the PostgreSQL server using the configured PostgreSQL password.
3. In the left-side panel, expand:

Servers → PostgreSQL → Databases

2.2 Create a New Database

1. Right-click on **Databases**.
2. Select **Create → Database...**
3. In the **Database** field, enter: **exp7_db**
4. Keep the default owner as: **postgres**
5. Click **Save**.

The database `exp7_db` is now created successfully.



Step 3: Configure PostgreSQL

Open the following file in the Spring Boot project:

`src/main/resources/application.properties`

Add the following database configuration in **application.properties**

```
spring.application.name=exp7

spring.datasource.url=jdbc:postgresql://localhost:5432/exp7_db
spring.datasource.username=postgres
spring.datasource.password=YOUR_DATABASE_PASSWORD

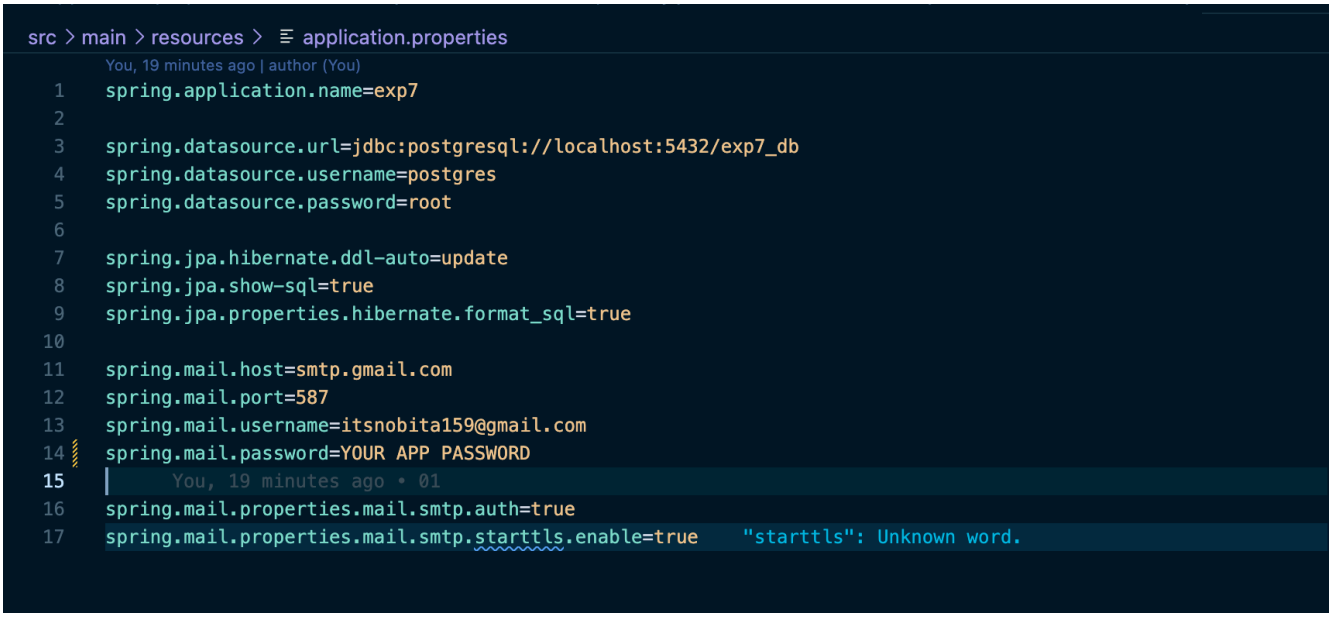
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Step 4: Configure Gmail SMTP

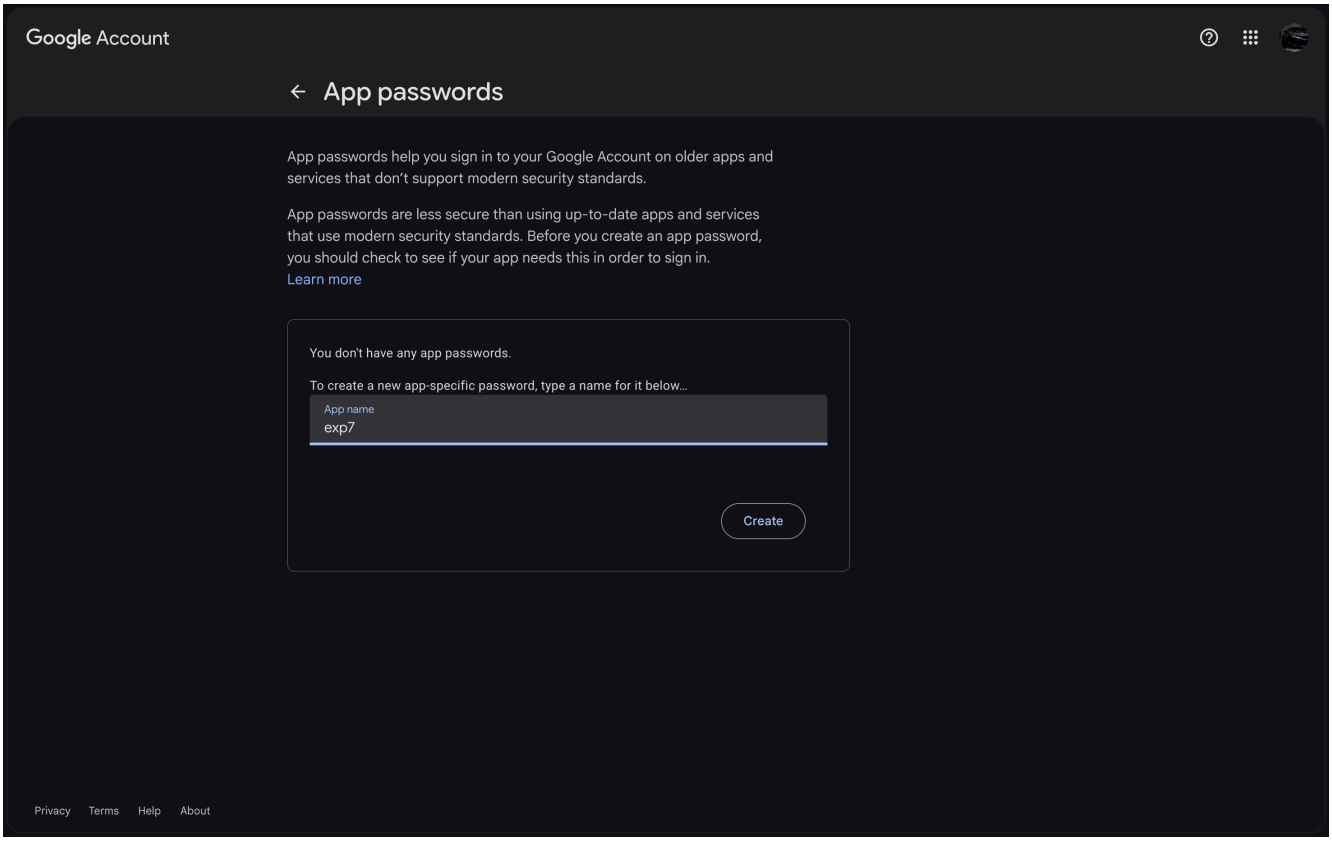
Also add the following to the application.properties to configure gmail SMTP after above code.

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=YOUR_GMAIL_ADDRESS
spring.mail.password=YOUR_GMAIL_APP_PASSWORD

spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```



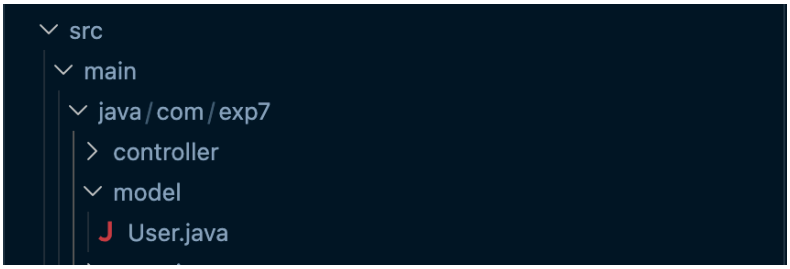
Make sure your Gmail account has turned on 2-Step Verification and a Gmail App Password was generated here **https://myaccount.google.com/apppasswords**. The App Password was used for SMTP authentication instead of the normal Gmail password.



Step 5: Create the User Entity: exp7/src/main/java/com/exp7/model/User.java

Create the Folder named **model**, then create the file in that model folder named as **User.java**

The User entity contains userId, name, email, password, and role. The table is named users because user caused a PostgreSQL reserved-keyword error.



model/User.java:

```
package com.exp7.model;

import jakarta.persistence.Table;

import jakarta.persistence.Entity;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;

@Entity

@Table(name = "users")

public class User {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long userId;


    private String name;

    private String email;

    private String password;

    private String role;

    public User() {

    }

    public Long getUserId() {

        return userId;

    }

    public void setUserId(Long userId) {

        this.userId = userId;

    }

    public String getName() {

        return name;

    }

    public void setName(String name) {

        this.name = name;

    }

    public String getEmail() {

        return email;

    }

    public void setEmail(String email) {

        this.email = email;

    }
```

```
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getRole() {
        return role;
    }
    public void setRole(String role) {
        this.role = role;
    }
}
```

Step 6: Create the Repository

Create the **repository** folder, add the **UserRepository.java** in that repository folder:



UserRepository.java:

```
package com.exp7.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.exp7.model.User;

public interface UserRepository extends JpaRepository<User, Long> {
}
```

Step 7: Create the User Service

Create the **service** folder, add the **UserService.java** in that service folder:



UserService.java:

```
package com.exp7.service;

import org.springframework.stereotype.Service;
import com.exp7.model.User;
import com.exp7.repository.UserRepository;

@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User registerUser(User user) {
        return userRepository.save(user);
    }
}
```

Step 8: Create the Email Service

Create the **service** folder, add the **EmailService.java** in that service folder:



EmailService.java:

```
package com.exp7.service;

import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    private final JavaMailSender mailSender;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    public void sendConfirmationEmail(String to, String name) {

        SimpleMailMessage message = new SimpleMailMessage();

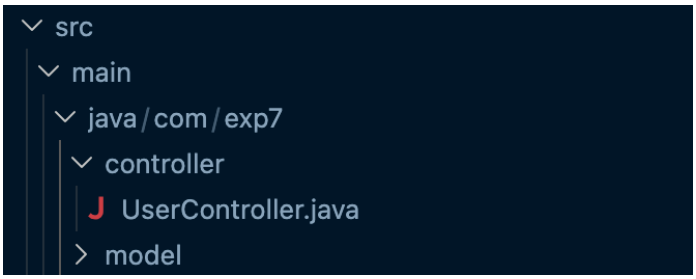
        message.setTo(to);
        message.setSubject("Registration Successful");
        message.setText(
```

```
                "Hello " + name + "\n\n" +
                "Your account has been successfully created.\n\n" +
                "Thank you for registering."
            );

            mailSender.send(message);
        }
    }
```

Step 9: Create the Signup Controller

Create the **controller** folder, add the **UserController.java** in that controller folder:



UserController.java:

```
package com.exp7.controller;

import org.springframework.web.bind.annotation.*;
import com.exp7.model.User;
import com.exp7.service.EmailService;
import com.exp7.service.UserService;

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;
    private final EmailService emailService;

    public UserController(UserService userService, EmailService emailService) {
        this.userService = userService;
        this.emailService = emailService;
    }

    @PostMapping("/signup")
    public User signup(@RequestBody User user) {

        User savedUser = userService.registerUser(user);

        emailService.sendConfirmationEmail(
                savedUser.getEmail(),
                savedUser.getName()
        );

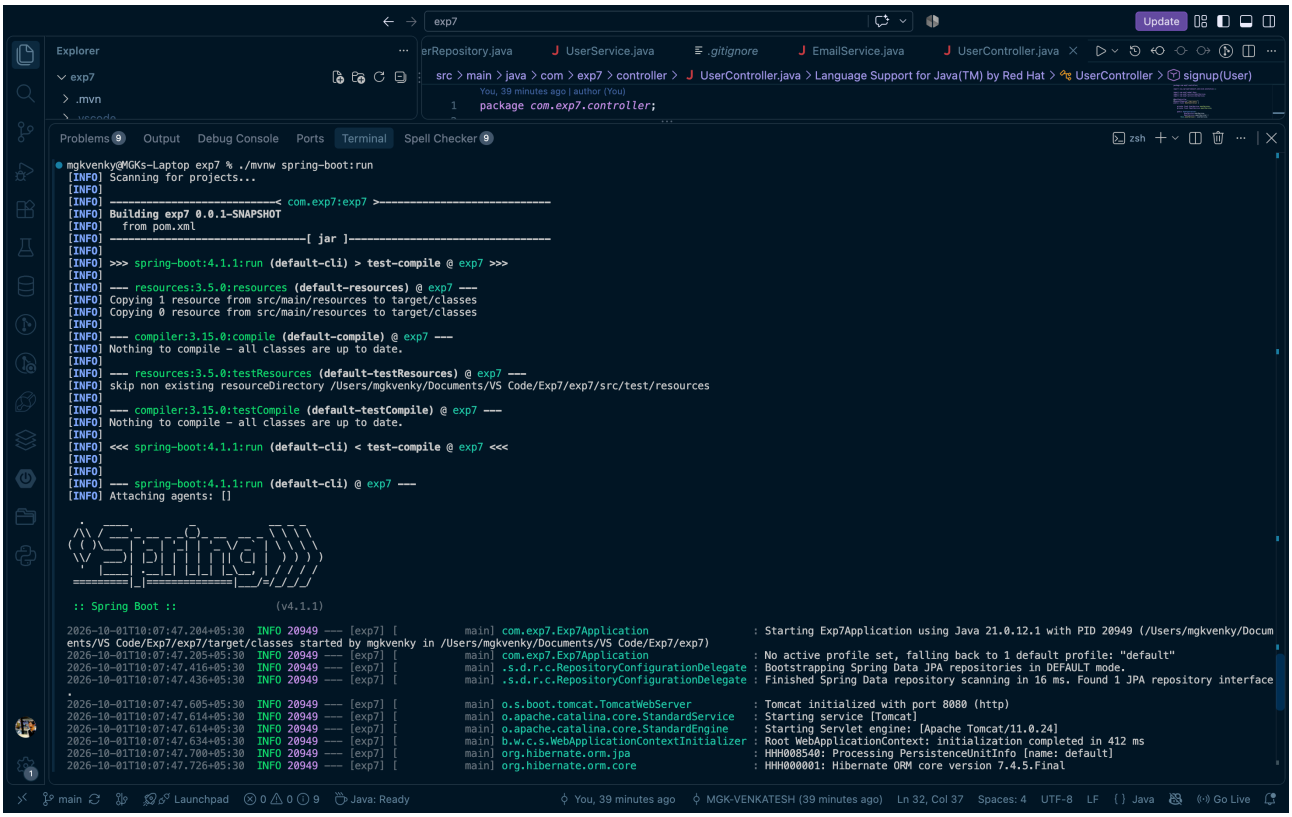
        savedUser.setPassword(null);

        return savedUser;
    }
}
```

Step 10: Run the Application

```
./mvnw spring-boot:run
```

The Spring Boot application started successfully on port 8080 and connected to PostgreSQL.



Step 11: Test the Signup API

Postman was used to test the signup endpoint. Open postman app, create the POST request with link

```
POST http://localhost:8080/api/users/signup
```

Header:

```
Content-Type: application/json
```

Request body:

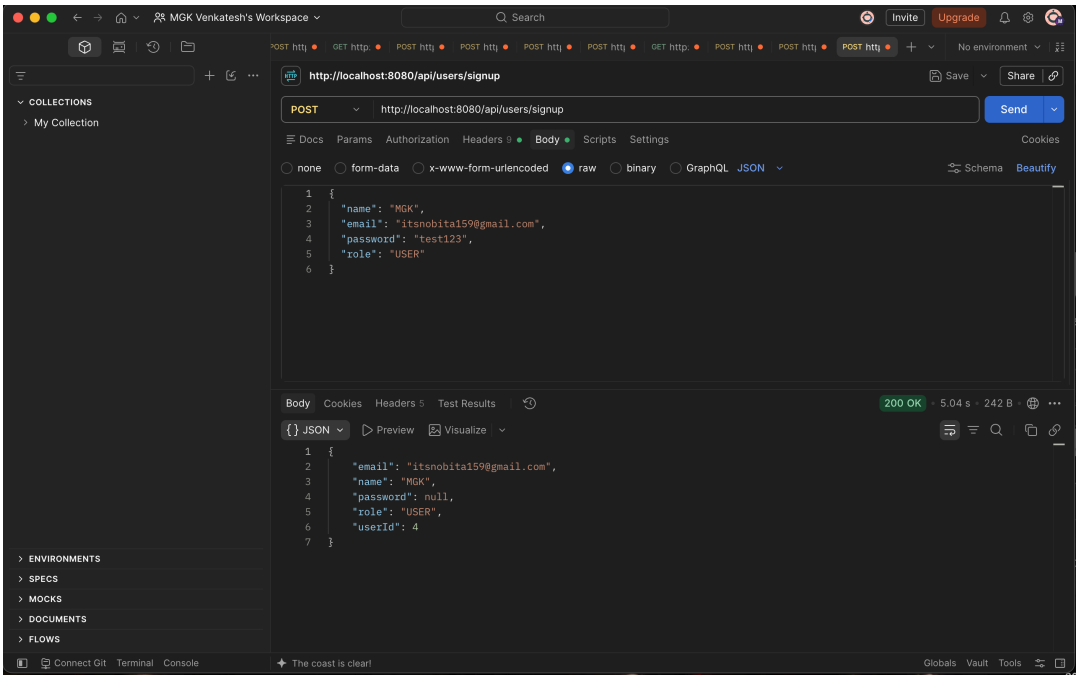
```
{
  "name": "YOUR_NAME",
  "email": "YOUR_EMAIL@gmail.com",
  "password": "test123",
  "role": "USER"
}
```

Then after Click on Send, you should get the following output of 200 OK.

Step 12: Verify API Response

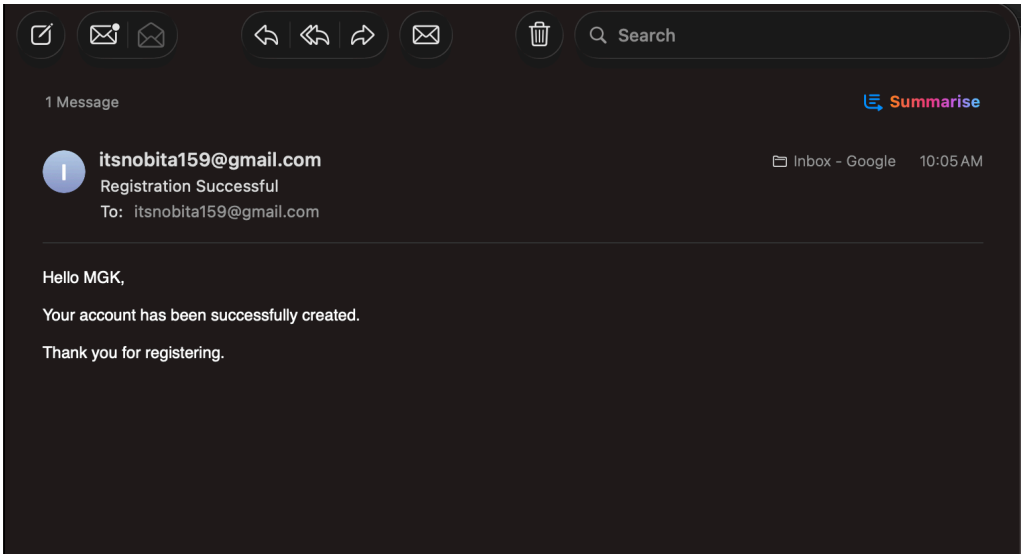
The request returned HTTP 200 OK. The user was stored in PostgreSQL and a userId was generated. The password was removed from the response.

```
{
  "email": "YOUR_EMAIL@gmail.com",
  "name": "YOUR_NAME",
  "password": null,
  "role": "USER",
  "userId": 4
}
```



Step 13: Verify the Confirmation Email

The confirmation email was successfully received by the registered Gmail account.



5. Working Process / Flow

```
Signup request
      ↓
UserController
      ↓
UserService
      ↓
Save user in PostgreSQL
      ↓
EmailService
      ↓
JavaMailSender
      ↓
Gmail SMTP
      ↓
Confirmation email received
```

6. Result

Thus, the Email Notification system was successfully implemented using Spring Boot, JavaMailSender, Gmail SMTP, PostgreSQL, and Postman. The confirmation email was successfully delivered after user signup.